

ARCHITECTURE PLAN

Fill 1-3 + 11 before any code | Fill 4-10 per feature as you build

Project

Date started

Date finished

Before writing any code, answer these two questions:

1. What data does this app need to REMEMBER?

2. What can be COMPUTED from that data? (never store computed values)

1 PRODUCT DESCRIPTION

Plain English only - no tech terms. What does it do, who uses it, what problem does it solve?

Question	Answer
What does it do?	
Who uses it?	
What problem does it solve?	

2 PAGES AND ROUTES

New screen = layout changes completely + back button makes sense + its own URL. Otherwise it's a state change.

Display data only -> server component | Forms / interactions / state -> client component

Route	Page Name	What it shows	Server or Client?
/			

3 DATA SHAPE

One block per entity. What info does ONE item need to hold?

Entity 1

Field name	Data type	Field name	Data type

Entity 2

Field name	Data type	Field name	Data type

Entity 3

Field name	Data type	Field name	Data type

4 STATE INVENTORY

Can I COMPUTE this from existing state? Yes -> never store it.

True state | Local UI state | Derived (never stored) | Server state | Global / Context (3+ levels)

State	Kind	Owned by

5 COMPONENT TREE

Write component/file names on the blank lines under each folder. This structure reflects standard Next.js project layout.

src /

app / Next.js file-based routing - one folder = one route

(routes) / route groups - group pages without affecting URL

| - _____

| - _____

| - _____

layout.tsx root layout - wraps every page (nav, fonts, providers)

page.tsx home page

loading.tsx automatic loading UI while page fetches

error.tsx automatic error boundary for this route

features / *one folder per feature - self-contained vertical slices*

feature 1: _____ /

components / *UI only for this feature*

| - _____

| - _____

| - _____

| - _____

| - _____

hooks / *custom hooks for this feature*

| - _____

| - _____

| - _____

utils / *helpers scoped to this feature*

| - _____

| - _____

types.ts *TypeScript types for this feature*

index.ts *public API - only export what other features need*

feature 2: _____ /

components / *UI only for this feature*

| - _____

| - _____

| - _____

| - _____

| - _____

hooks / *custom hooks for this feature*

| - _____

| - _____

| - _____

utils / *helpers scoped to this feature*

| - _____

- _____
types.ts <i>TypeScript types for this feature</i>
index.ts <i>public API - only export what other features need</i>
feature 3: _____ /
components / <i>UI only for this feature</i>
- _____
- _____
- _____
- _____
- _____
hooks / <i>custom hooks for this feature</i>
- _____
- _____
- _____
utils / <i>helpers scoped to this feature</i>
- _____
- _____
types.ts <i>TypeScript types for this feature</i>
index.ts <i>public API - only export what other features need</i>
feature 4: _____ /
components / <i>UI only for this feature</i>
- _____
- _____
- _____
- _____
- _____
hooks / <i>custom hooks for this feature</i>
- _____
- _____
- _____

utils / <i>helpers scoped to this feature</i>
- _____
- _____
types.ts <i>TypeScript types for this feature</i>
index.ts <i>public API - only export what other features need</i>
feature 5: _____ /
components / <i>UI only for this feature</i>
- _____
- _____
- _____
- _____
- _____
hooks / <i>custom hooks for this feature</i>
- _____
- _____
- _____
utils / <i>helpers scoped to this feature</i>
- _____
- _____
types.ts <i>TypeScript types for this feature</i>
index.ts <i>public API - only export what other features need</i>
feature 6: _____ /
components / <i>UI only for this feature</i>
- _____
- _____
- _____
- _____
- _____
hooks / <i>custom hooks for this feature</i>
- _____

- _____
- _____
utils / <i>helpers scoped to this feature</i>
- _____
- _____
types.ts <i>TypeScript types for this feature</i>
index.ts <i>public API - only export what other features need</i>
shared / <i>truly reusable code - used by 2+ features</i>
ui / <i>generic UI components (Button, Modal, Input...)</i>
- _____
- _____
- _____
- _____
- _____
hooks / <i>reusable hooks (useDebounce, useLocalStorage...)</i>
- _____
- _____
- _____
utils / <i>pure helper functions (formatDate, cn...)</i>
- _____
- _____
types / <i>shared TypeScript types and interfaces</i>
- _____
- _____
constants / <i>app-wide constants (routes, config, enums)</i>
- _____
- _____
lib / <i>third-party config and initialisation</i>
<i>e.g. db.ts, auth.ts, stripe.ts, prisma.ts</i>
- _____

- _____
- _____
store / <i>global state - only if needed (Zustand, Redux...)</i>
<i>e.g. useAuthStore.ts, useCartStore.ts</i>
- _____
- _____
- _____
styles / <i>global CSS, Tailwind base, design tokens</i>
<i>e.g. globals.css, tokens.css</i>
- _____
- _____
public / <i>static assets served at / (images, fonts, icons)</i>
<i>e.g. logo.svg, og-image.png</i>
- _____
- _____

5b COMPONENT RESPONSIBILITIES

One job per component. If 'what it does' contains 'and' - split it. Start with a verb.

Component	What it does (verb first)	UI elements inside

Component	What it does (verb first)	UI elements inside

6 CONTEXT CANDIDATES - ONLY IF NEEDED

Only use Context if state travels 3+ levels deep. 1-2 levels -> just pass props.

State	Context name	Why it needs Context

7 DATA FLOW MAP

Data flows DOWN (props). Events flow UP (callbacks named onX).

Component	Props it receives (data down)	Callbacks it receives (events up)

8 HANDLER INVENTORY

Naming: handleX in the component | passed down as onX prop

Handler	What it does	Lives in

9 API / DATA FETCHING PLAN - ONLY IF NEEDED

Always plan three states: loading | success | error. Never just the happy path.

Data	Fetches where	Method	Loading?	Error handling

10 CHECKLIST - BEFORE WRITING EACH FEATURE

☐	Do I know what state I need?
☐	Is this true state, derived state, or server state?
☐	Which component is the lowest common ancestor - who owns the state?
☐	Am I mutating or copying? (always copy - spread, map, filter)
☐	What props go down? What callbacks go up?
☐	Does any state need Context, or can I just pass props?
☐	Is this component server or client?
☐	Does this logic belong in features/ or shared/?
☐	Have I handled loading state?
☐	Have I handled error state?

11 BUILD ORDER

Always static before interactive. Never start a new feature until the current one works end to end.

User must act (click / type / submit) -> Interactive | It just appears on screen -> Static

#	Feature	Static or Interactive?	Depends on

11.5 GIT STRATEGY

Branch: feature/name fix/name refactor/name chore/name

Commit: feat: ... fix: ... refactor: ... docs: ... style: ... chore: ...

Feature	Branch name	Commit message

Feature	Branch name	Commit message

11.6 COMMIT CHECKLIST

☐	Does the feature work end-to-end?
☐	Are there any console errors?
☐	Is the commit message formatted as type: description (present tense)?
☐	Am I on the correct branch?

Notes NOTES & DECISIONS

Decisions, open questions, confusing parts, anything that doesn't fit above.

BEFORE ANY CODE	1. Product Description 2. Pages & Routes 3. Data Shape 11. Build Order
PER FEATURE	4. State 5. Component Tree 5b. Responsibilities 7. Data Flow 8. Handlers 10. Checklist
ONLY IF NEEDED	6. Context (prop drilling) 9. API Plan (fetching data)

EXTRA NOTES

Decisions, open questions, confusing parts, anything that does not fit above.

EXTRA NOTES

Decisions, open questions, confusing parts, anything that does not fit above.

5 COMPONENT TREE - CONTINUED

Write component/file names on blank lines under each folder. Standard Next.js project layout.

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

| - _____

- _____
- _____
- _____

4 STATE INVENTORY - CONTINUED

Can I COMPUTE this from existing state? Yes -> never store it.

True state | Local UI state | Derived (never stored) | Server state | Global / Context (3+ levels)

[illegible]

7 DATA FLOW MAP - CONTINUED

Data flows DOWN (props). Events flow UP (callbacks named onX).

[illegible]

8 HANDLER INVENTORY - CONTINUED

Naming: handleX in the component | passed down as onX prop

[illegible]

11 BUILD ORDER - CONTINUED

Always static before interactive. Never start a new feature until the current one works end to end.

User must act (click / type / submit) -> Interactive | It just appears on screen -> Static

[illegible]

11.5 GIT STRATEGY - CONTINUED

[illegible]

5b COMPONENT RESPONSIBILITIES - CONTINUED

One job per component. If 'what it does' contains 'and' - split it. Start with a verb.

[illegible]